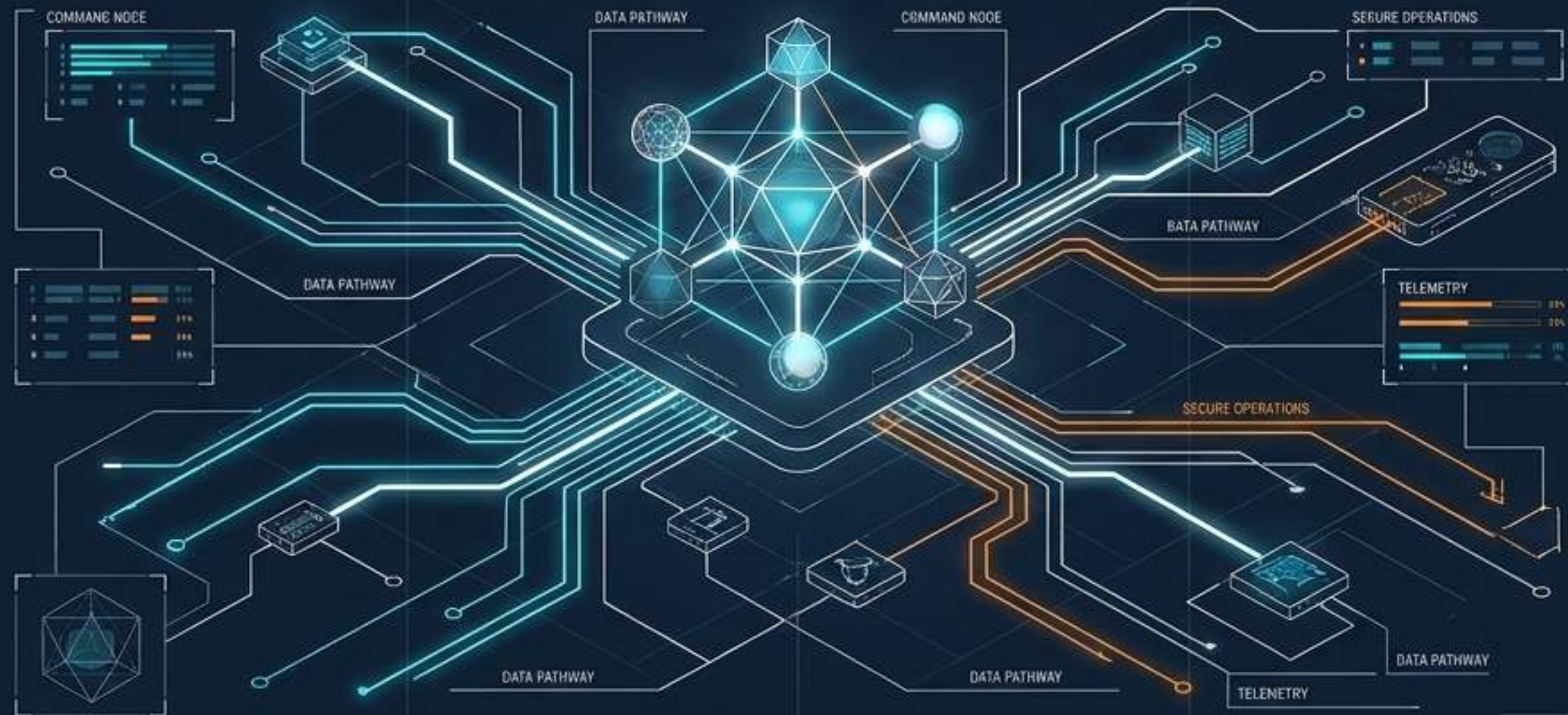


Operational Orchestration GUI

High-Level Architecture & System Blueprint



The Paradigm: Operational Command & Control

This is NOT a CRUD application.

Traditional CRUD



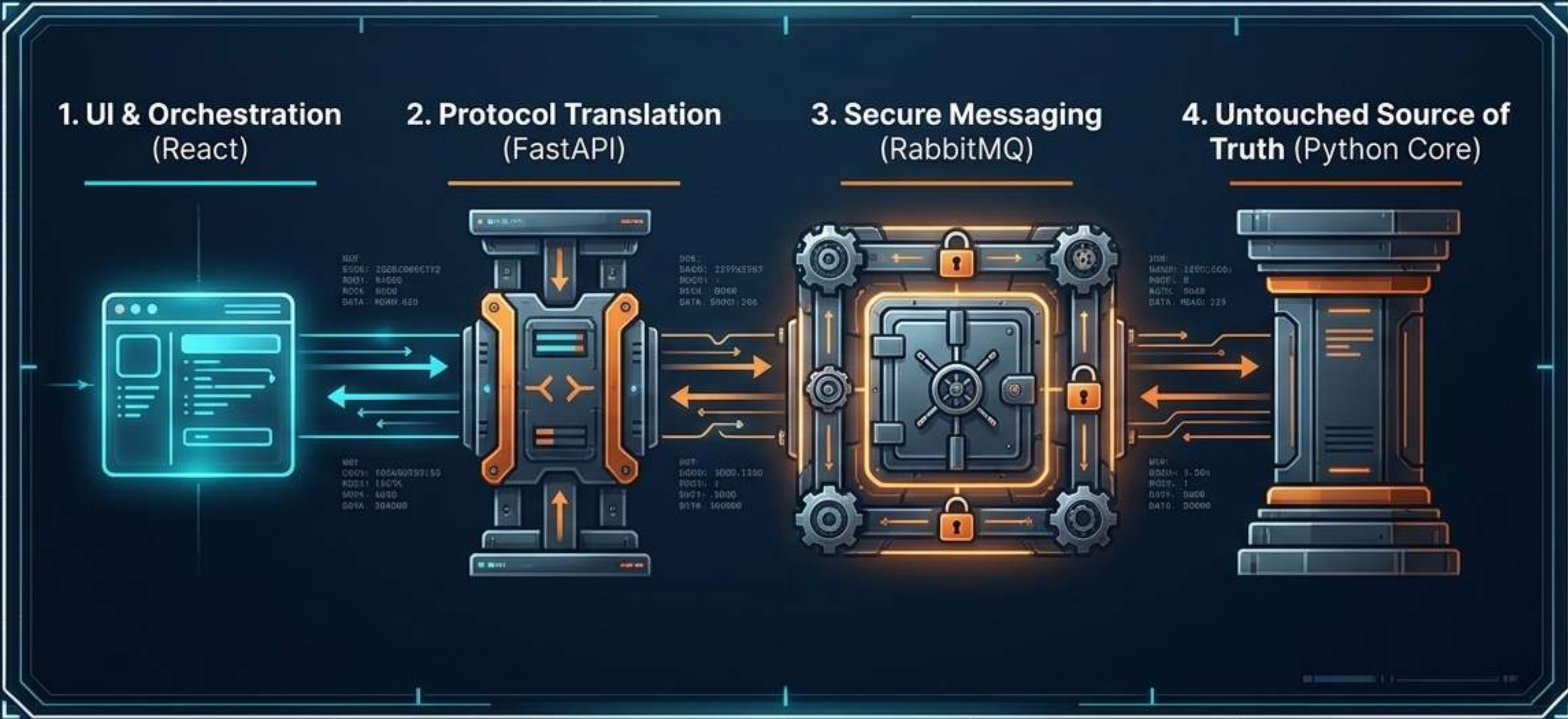
- Instant responses.
- Frontend logic.
- Synchronous data.

Operational Command & Control



- ✓ Backend-driven state & logic.
- ✓ Thin UI acting solely as an orchestration layer.
- ✓ Zero disruption to legacy backend systems.

Master Architecture Blueprint



Frontend Selection: React + Vite

Evaluation Dimensions: Simplicity | Maintainability | Operational Suitability | Learning Curve

 Angular	 Next.js	  React + Vite
Too heavy; extremely steep learning curve.	Introduces Server-Side Rendering (SSR) complexity that is not currently needed.	Highest simplicity, massive ecosystem, ideal for single-page operational tooling, junior-friendly learning curve.

Final Decision

React + Vite. Optimized for internal UI, zero SEO requirements, highly maintainable. Migrating to Next.js remains a viable future option if SSR is required.



Backend-For-Frontend (BFF): FastAPI

Next.js Server (Node.js)

- Pros: Unified JavaScript stack for the web layer.
- Cons: Introduces costly context switching from our existing Python infrastructure.

FastAPI (Python)

- Pros: Native Python ecosystem, shared models/validation with Core, flawless RabbitMQ integration.
- Cons: Introduces an additional deployable service.

Final Decision: FastAPI

Architectural alignment and shared Python models outweigh the cost of an extra service.

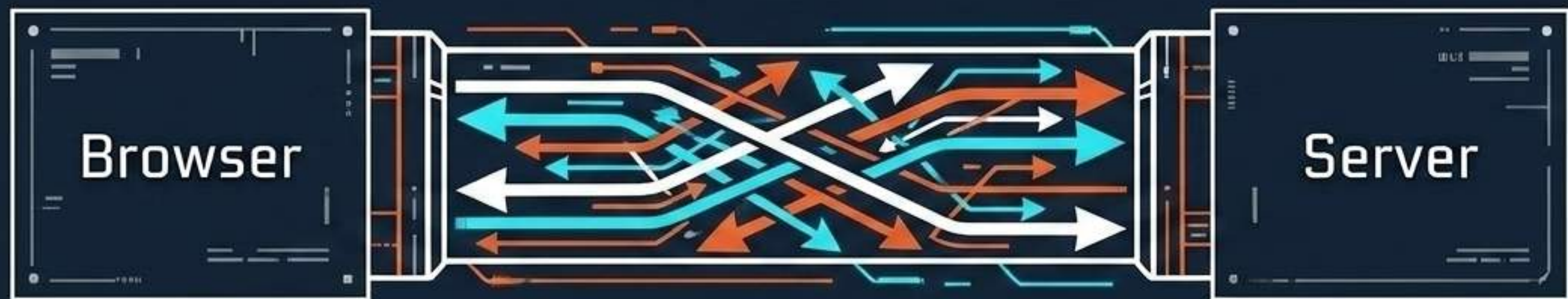
Communication Analysis (1/3): REST Only



Architectural Assessment: Inadequate

While perfect for discrete commands, relying on REST polling for streaming logs creates massive network spam, brittle error handling, and a laggy operational UX.

Communication Analysis (2/3): REST + WebSockets

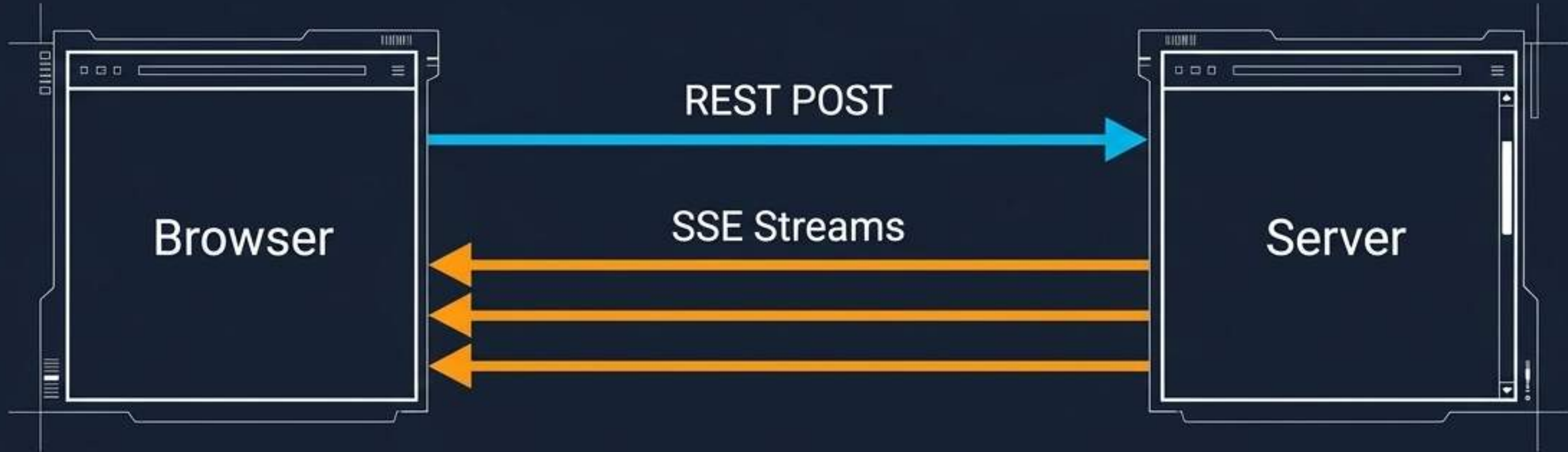


Bidirectional, persistent,
stateful connection.

Architectural Assessment: Overkill

Introduces massive operational complexity. Reconnect logic, state-synchronization risks, and load balancing overhead present high risk for a soft-real-time system.

Final Decision (3/3): REST Commands + SSE Telemetry



Standard HTTP POST for actionable commands.
Server-Sent Events (SSE) for one-way downstream telemetry.

Architectural Assessment: Optimal Match

Perfectly aligns with asymmetric operational traffic. Provides highly stable, automatic reconnects for streaming logs with a fraction of WebSocket complexity.

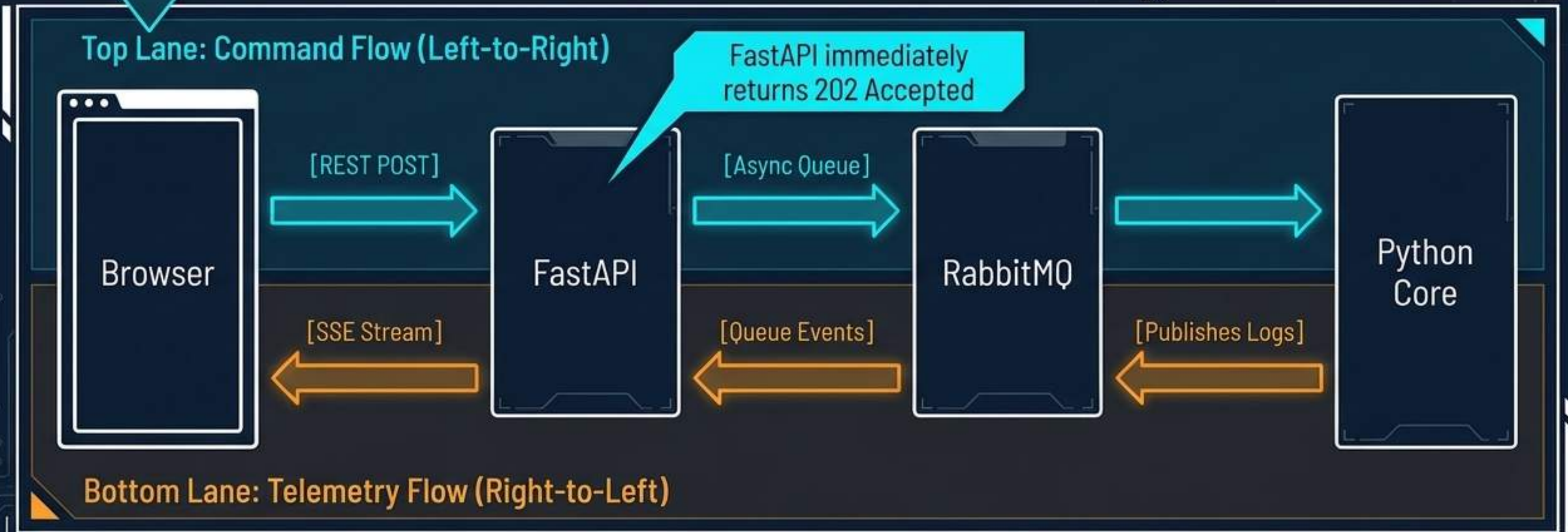
Deployment Flexibility



Concept		Value
1	Concept	Network-Independent Architecture.
2	Mechanism	RabbitMQ bridges strictly enforced internal security zones. Eliminates brittle, point-to-point HTTP links.
3	Result	Future deployments across different network zones require absolutely zero application code changes.

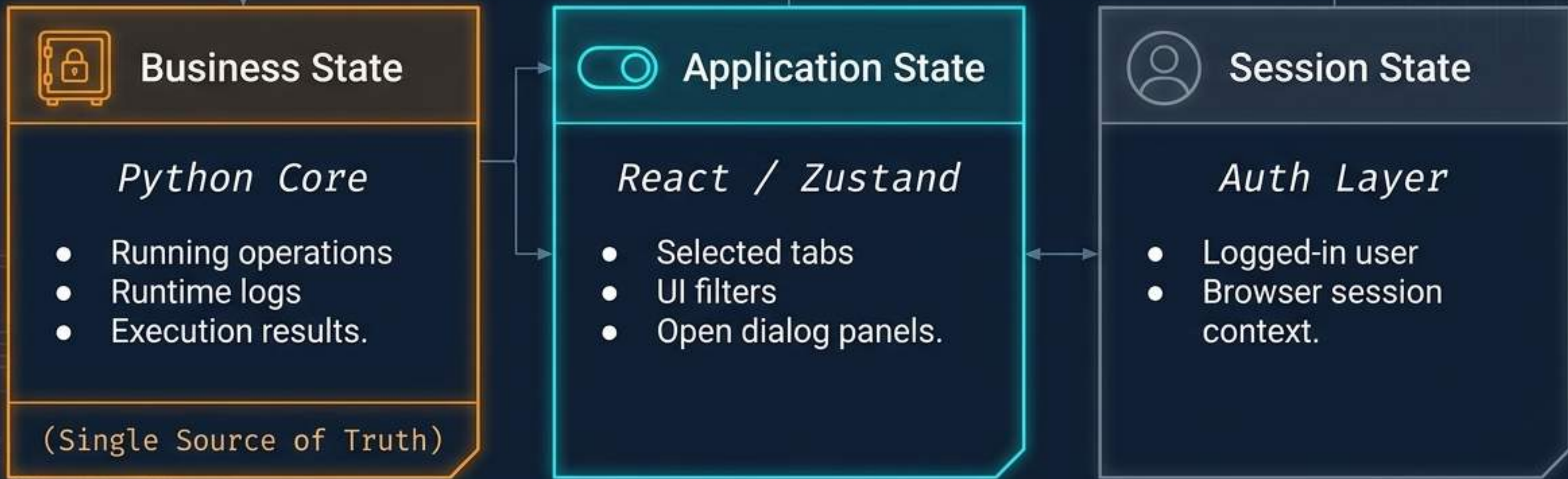
The Orchestration Loop: Mechanical Flows

Key Principle: The UI never blocks waiting for execution. It strictly issues commands (Top Lane) and listens to results (Bottom Lane).

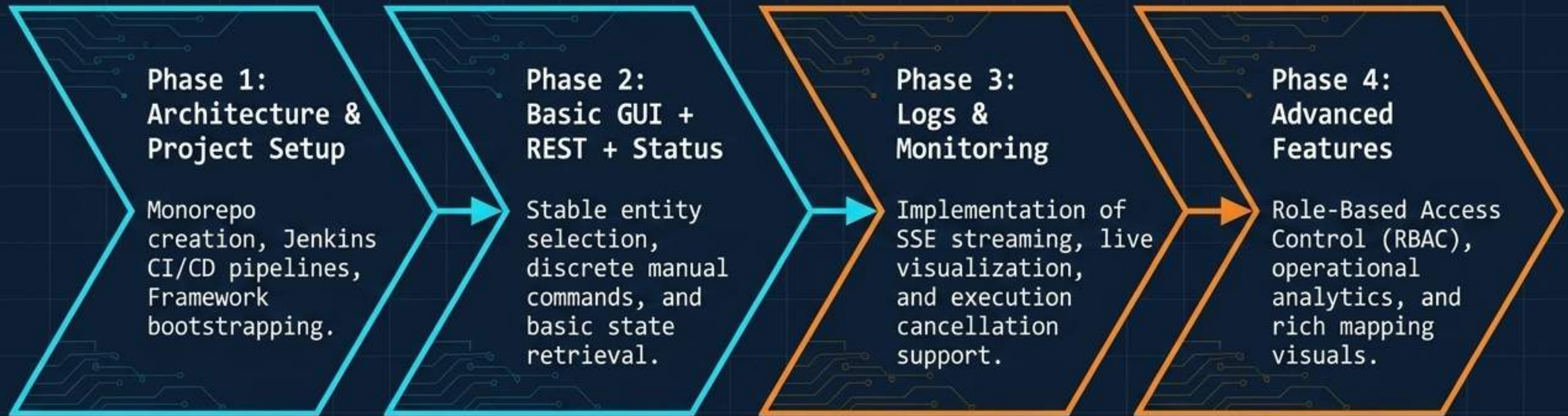


State Management Strategy

The Golden Rule: A browser refresh must perfectly and completely restore the **Business State** from the backend without manual duplication.



Incremental Delivery Phases



AI Development Guardrails

AI is a code accelerator, not a system architect.



AI Writes

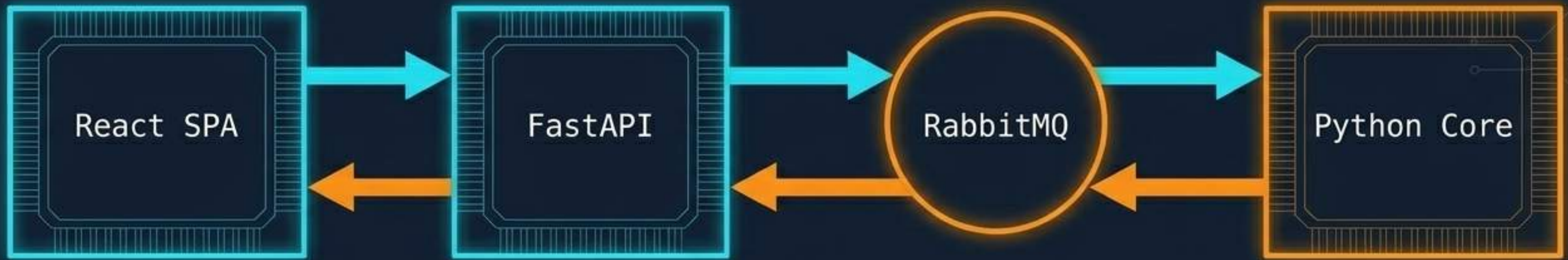
- UI scaffolding
- Standard boilerplate
- Material UI forms
- TypeScript types



Humans Own

- System architecture
- Asynchronous lifecycles
- Backend API contracts
- State ownership

Final Architecture Summary



Simple

Minimal custom
framework logic.

Asynchronous

Built explicitly for
long-running workflows.

Maintainable

Strict boundary
separation.

**Network-
Independent**

Bridged securely by
messaging queues.

Immediate Next Steps

- ✓ Finalize Core Business Flows
- ✓ Create UML Component Diagrams
- ✓ Define Strict RabbitMQ Contracts
- ✓ Create Monorepo & Configure Jenkins CI/CD
- ✓ Bootstrap React + Vite & FastAPI Frameworks
- ✓ Implement First End-to-End Mechanical Flow

Reference: Technology Glossary

SPA	Single Page Application (loads once, dynamically updates UI without page reloads).	WebSocket	Bidirectional persistent connection (Avoided due to complexity).
Vite	Modern, lightning-fast build tool for frontend frameworks.	RabbitMQ	Secure asynchronous message broker (The internal network queue).
FastAPI	High-performance Python framework for building APIs.	SSR	Server-Side Rendering (Rendering HTML on the server - Not required).
BFF	Backend-For-Frontend (an API translation layer built specifically for the UI).	Monorepo	Single Git repository holding multiple associated projects.
REST	Standard HTTP request pattern utilized for discrete Commands.	State	The memory/data condition of the application at a given runtime.
SSE	Server-Sent Events (One-way downstream data stream for logs).	CI/CD	Continuous Integration / Continuous Deployment pipeline.